Course Title: Cryptography and Network Security Lab	**Course Code**: BCSE309P
Faculty: Prof. Karthika V	Slot: *L15+L16*
Regno: *23BCE1099*	Name: *Arkita Barua*

EXERCISE 9- DSS

Code

Sender

```
import java.io.ObjectOutputStream;
import java.net.Socket;
import java.math.BigInteger;
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.Scanner;

public class sender_DSS {

    public static void main(String[] args) throws Exception {

        SecureRandom rand = new SecureRandom();
        Scanner sc = new Scanner(System.in);

        System.out.println("===== DSS SENDER =====");

        // Generate q
        BigInteger q = BigInteger.probablePrime(160, rand);

        // Generate p
        BigInteger p, temp;
        do {
            temp = new BigInteger(352, rand);
            p = temp.multiply(q).add(BigInteger.ONE);
        } while (!p.isProbablePrime(50));

        // Generate g
        BigInteger g, h;
        do {
            h = new BigInteger(350, rand);
            g = h.modPow((p.subtract(BigInteger.ONE)).divide(q), p);
        } while (g.compareTo(BigInteger.ONE) <= 0);

        System.out.println("\nGlobal Public Key Components:");
```

```java
        System.out.println("p: " + p);
        System.out.println("q: " + q);
        System.out.println("g: " + g);

        // Keys
        BigInteger x = new BigInteger(159, rand); // private
        BigInteger y = g.modPow(x, p);            // public

        System.out.println("\nUser's Private Key (x): " + x);
        System.out.println("User's Public Key (y): " + y);

        // Input message
        System.out.print("\nEnter the message: ");
        String message = sc.nextLine();

        // Hash
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        BigInteger hash = new BigInteger(1, md.digest(message.getBytes()));
        System.out.println("Hash Value (SHA-256): " + hash);

        // Choice for invalid test case
        System.out.print("\nDo you want to send the correct message? (y/n): 
");
        char choice = sc.next().charAt(0);

        if (choice == 'n') {
            x = x.add(BigInteger.TWO); // deliberately corrupt
        }

        // Signature
        BigInteger[] sig = generateSignature(message, p, q, g, x);
        BigInteger r = sig[0];
        BigInteger s = sig[1];

        System.out.println("\nSignature: [r: " + r + ", s: " + s + "]");

        // Send data
        Socket socket = new Socket("localhost", 5000);
        ObjectOutputStream output = new 
ObjectOutputStream(socket.getOutputStream());

        System.out.println("\nSending to client: p,q,g,y,r,s,Message");

        output.writeObject(p);
        output.writeObject(q);
        output.writeObject(g);
        output.writeObject(y);
        output.writeObject(message);
```

```
        output.writeObject(r);
        output.writeObject(s);

        System.out.println("Sent Successfully");

        output.close();
        socket.close();
    }

    public static BigInteger[] generateSignature(String msg, BigInteger p,
BigInteger q,
                                                 BigInteger g, BigInteger x)
throws Exception {

        SecureRandom rand = new SecureRandom();

        MessageDigest md = MessageDigest.getInstance("SHA-256");
        BigInteger hash = new BigInteger(1, md.digest(msg.getBytes()));

        BigInteger k, r, s;

        do {
            k = new BigInteger(159, rand);

            r = g.modPow(k, p).mod(q);

            BigInteger kInv = k.modInverse(q);
            s = (kInv.multiply(hash.add(x.multiply(r)))).mod(q);

        } while (r.equals(BigInteger.ZERO) || s.equals(BigInteger.ZERO));

        return new BigInteger[]{r, s};
    }
}
```

Receiver

```
import java.io.ObjectInputStream;
import java.net.ServerSocket;
import java.net.Socket;
import java.math.BigInteger;
import java.security.MessageDigest;

public class receiver_dss {

    public static void main(String[] args) throws Exception {

        System.out.println("===== DSS RECEIVER =====");
```

```java
        System.out.println("Server waiting on port 5000...\n");

        ServerSocket server = new ServerSocket(5000);
        Socket connection = server.accept();

        ObjectInputStream input = new
ObjectInputStream(connection.getInputStream());

        System.out.println("Client connected!\n");

        // Receive all components
        BigInteger p = (BigInteger) input.readObject();
        BigInteger q = (BigInteger) input.readObject();
        BigInteger g = (BigInteger) input.readObject();
        BigInteger y = (BigInteger) input.readObject();
        String msg = (String) input.readObject();
        BigInteger r = (BigInteger) input.readObject();
        BigInteger s = (BigInteger) input.readObject();

        System.out.println("Received from Sender: p,q,g,y,r,s,Message");
        System.out.println("Message: " + msg);

        // Verification steps
        boolean result = verifySignature(p, q, g, y, msg, r, s);

        if (result) {
            System.out.println("\nSuccessful Verified Signature");
        } else {
            System.out.println("\nIncorrect Signature !!!");
        }

        input.close();
        connection.close();
        server.close();
    }

    public static boolean verifySignature(BigInteger p, BigInteger q,
BigInteger g,
                                          BigInteger y, String msg,
                                          BigInteger r, BigInteger s) throws
Exception {

        // Step 1: Check bounds
        if (r.compareTo(BigInteger.ZERO) <= 0 || r.compareTo(q) >= 0 ||
            s.compareTo(BigInteger.ZERO) <= 0 || s.compareTo(q) >= 0) {
            return false;
        }
```

```java
        // Step 2: Hash
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        BigInteger hash = new BigInteger(1, md.digest(msg.getBytes()));

        // Step 3: w = s^-1 mod q
        BigInteger w = s.modInverse(q);

        // Step 4: u1 and u2
        BigInteger u1 = hash.multiply(w).mod(q);
        BigInteger u2 = r.multiply(w).mod(q);

        System.out.println("\nVerification:");
        System.out.println("w: " + w);
        System.out.println("u1: " + u1);
        System.out.println("u2: " + u2);

        // Step 5: v calculation
        BigInteger part1 = g.modPow(u1, p);
        BigInteger part2 = y.modPow(u2, p);
        BigInteger v = part1.multiply(part2).mod(p).mod(q);

        System.out.println("v: " + v);
        System.out.println("\nr' (from Sender): " + r);
        System.out.println("v  (Calculated): " + v);

        return v.equals(r);
    }
}
```

Output [screenshot]

Test case 1: Signature Valid

Sender

```
PS E:\aa vit sem6\z_Crypto-lab\Ex9_DSS> javac sender_DSS.java
PS E:\aa vit sem6\z_Crypto-lab\Ex9_DSS> java sender_DSS
===== DSS SENDER =====

Global Public Key Components:
p: 7002317799584847114932900308027721244424292326444359695880927077380286974955974352122384229904650336430421474471952
1431378389385076180070557341050409311 93
q: 946049190511037425546555775427809039341516662551
g: 7360594751054887276106021067265955126243450311740855403650981479721661178197620934302785984083392691800117917872567
637706786446852560276908094130853052 27

User's Private Key (x): 1122534589666841124170612875297249125420028039 14
User's Public Key (y): 3102052342806199884031539009755357499434352425104225312008900456875212138691405376676719459965 7
962327416713589536386233823244232083392050245799010604815 51

Enter the message: Arkita
Hash Value (SHA-256): 102686277669769519557681829903588009364919249871713305721017414859188116387534

Do you want to send the correct message? (y/n): y

Signature: [r: 660013237103523947437177366036060370133953524887, s: 879781890351478787322104175908434780905815488099]

Sending to client: p,q,g,y,r,s,Message
Sent Successfully
PS E:\aa vit sem6\z_Crypto-lab\Ex9_DSS>
```

Receiver

```
PS E:\aa vit sem6\z_Crypto-lab\Ex9_DSS> javac receiver_dss.java
PS E:\aa vit sem6\z_Crypto-lab\Ex9_DSS> java receiver_dss
===== DSS RECEIVER =====
Server waiting on port 5000...

Client connected!

Received from Sender: p,q,g,y,r,s,Message
Message: Arkita

Verification:
w: 559084299517769222785758422531152304090273999608
u1: 409796073146097596607781248784903276106654793384
u2: 649391361613475604558377646187392462499701642 35
v: 660013237103523947437177366036060370133953524887

r' (from Sender): 660013237103523947437177366036060370133953524887
v  (Calculated): 660013237103523947437177366036060370133953524887

Successful Verified Signature
PS E:\aa vit sem6\z_Crypto-lab\Ex9_DSS>
```

Test Case 2: Signature Invalid

Sender

```
PS E:\aa vit sem6\z_Crypto-lab\Ex9_DSS> java sender_DSS
===== DSS SENDER =====

Global Public Key Components:
p: 37708407580662516344655180293967599038289798561837949487404030393683510323121934860730584655824856505563120136963884
64922517145699693009242253616356026947
q: 132012533372074564374443405354585106868713586169 3
g: 150782780107241912587546805470597365656528038694232838583964435666503240374066243027729036376786560341005887693441 4
341984070943842897470028536172146208572

User's Private Key (x): 293790557306071845953114535569676448512416111514
User's Public Key (y): 787449686017454793463337888777583924483463380270250276819448972628166548970300805298359567232 24
608188644988848058642310613244239548035694367914654716311 9

Enter the message: Arkita
Hash Value (SHA-256): 102686277669769519557681829903588009364919249871713305721017414859188116387534

Do you want to send the correct message? (y/n): n

Signature: [r: 104343779600967652764995750104391482629458128450 0, s: 960174834483008707341147502315493876553565372643]

Sending to client: p,q,g,y,r,s,Message
Sent Successfully
PS E:\aa vit sem6\z_Crypto-lab\Ex9_DSS>
```

Receiver

```
PS E:\aa vit sem6\z_Crypto-lab\Ex9_DSS> java receiver_dss
===== DSS RECEIVER =====
Server waiting on port 5000...

Client connected!

Received from Sender: p,q,g,y,r,s,Message
Message: Arkita

Verification:
w: 74578512141207377402613445514232288238653168687 4
u1: 106461353275826043788835447527138991803830939942 9
u2: 87853468179081988671753900988026766304671259252 4
v: 112845529537579141028895355445073041226234973735 7

r' (from Sender): 104343779600967652764995750104391482629458128450 0
v  (Calculated): 112845529537579141028895355445073041226234973735 7

Incorrect Signature !!!
PS E:\aa vit sem6\z_Crypto-lab\Ex9_DSS>
```